C PROGRAMMING FROM SCRATCH · EPISODE 11

Sizes and Signs

sizeof, short, long, signed and unsigned

KEY POINTS

- A byte is eight bits; sizeof gives a type's size in bytes, printed with %zu.
- The standard fixes only the order of sizes, so long and long double differ between platforms; stdint.h has exact widths.
- Signed types hold negative values, unsigned types don't; integers are signed by default, a plain char depends on the compiler.
- sizeof gives back a size_t, an unsigned type that %zu always matches.

01 Short and long

▶ VIDEO 00:35

`short int` is a smaller integer, `long int` a bigger one, and `long long int` bigger still. Two `long`s is the limit, and `int` can be left out: `short`, `long`, `long long`. Besides `float` and `double`, there's `long double` for more precision.

02 Bits and bytes

▶ VIDEO 01:25

A **bit** is a single 0 or 1, the smallest piece of data a computer can store. **Eight bits make one byte**, and C measures type sizes in bytes: a two-byte type has 16 bits.

03 Measuring with sizeof

▶ VIDEO 01:57

`sizeof(type)` gives the size in bytes. It's an operator and one of the 32 classic keywords, not a function.

C — C SOURCE

```c
printf("int: %zu\n", sizeof(int));   // prints: int: 4
```

Measured in Visual Studio (x64) with `sizes.c`:

CHAR	_BOOL	SHORT	INT	LONG	LONG LONG	FLOAT	DOUBLE	LONG DOUBLE
1	1	2	4	4	8	4	8	8

04 Sizes depend on the platform

▶ VIDEO 03:05

The standard fixes only the order: `char ≤ short ≤ int ≤ long ≤ long long` and `float ≤ double ≤ long double`.

TYPE	VISUAL STUDIO · WINDOWS	GCC · LINUX
`long`	4 bytes	8 bytes
`long double`	8 bytes	16 bytes

KEY IDEA

Don't memorize sizes: measure them with `sizeof`. For exact widths, `<stdint.h>` has `int8_t`, `int16_t`, `int32_t` and `int64_t` (exactly 8, 16, 32 and 64 bits).

05 Signed and unsigned

▶ VIDEO 04:38

They work only with `char` and the integer types. A **signed** type holds negative numbers, zero and positive numbers (a temperature, a bank balance); an **unsigned** type holds only zero and positive numbers (an age, a height). Print an `unsigned int` with `%u` (`unsigned-age.c` prints 25).

- `int` = `signed int`; `short`, `long` and `long long` are signed too. For unsigned, write `unsigned`.
- A plain `char` may be signed or unsigned, the compiler decides. When it matters, write `signed char` or `unsigned char`.

06 Why %zu?

▶ VIDEO 06:10

`sizeof` gives back a `size_t`: an unsigned integer type, because a size is never negative. Which one depends on the platform (`unsigned int`, `unsigned long` or `unsigned long long`, each with its own format); in Visual Studio on 64-bit Windows it's `typedef unsigned __int64 size_t;`. `%zu` **always matches** `size_t`.

CODE 3 files on GitHub `github.com/abitclearer/c-programming-from-scratch/tree/main/11-sizes-and-signs`